README.md

🔐 Security Toolkit

A professional, comprehensive GUI application for demonstrating and performing various cryptographic operations. Built with Python and Tkinter, featuring a modern dark-themed interface.

Python 3.x | GUI Tkinter | Img Pillow | License MIT

📖 Overview

Security Toolkit is an all-in-one cryptographic suite designed for both **practical use** ("Codes") and **education** ("Explanation").

Upon launching, users are greeted with a **Landing Page** to choose their mode:

1. 🛠️ **Codes**: Visualize and execute complexity algorithms (Encryption, Decryption, Hashing).
2. 📚 **Explanation**: Learn the concepts behind these algorithms with detailed text summaries and **schematic diagrams**.

The application features a clean, "hacker/cyberpunk" aesthetic (Dark Blue/Neon Green) and a highly responsive, compact interface.

✨ Features

📚 Explanation Mode (NEW!)

- **Visual Learning**: High-quality schematic diagrams for every algorithm (RSA, DES, S-DES, MD5, SHA-1, DSS, Diffie-Hellman, JWT).
- **Split View Layout**: Explanations on the left, diagrams on the right for easy reading.
- **Comprehensive Summaries**: Detailed text breaking down the history and mechanics of each method.

🔑 Public Key Cryptography

- **RSA Encryption System**:
  - Automated Prime & Key Generation (Public/Private keys).

    - Encrypt and Decrypt text messages.
- **Diffie-Hellman Key Exchange**:
    - Simulate secure key exchange between two parties (Alice & Bob).
    - **Attack Simulation**: Checkbox to simulate a Man-in-the-Middle (MITM) attack, demonstrating how key exchange fails when tampered with.
    - Visual diagram of the exchange process.
- **DSS (Digital Signature Standard)**:
    - Algorithm parameter visualization (p, q, g).
    - Sign and Verify messages to ensure authenticity.

🔒 Symmetric Key Cryptography

- **DES (Data Encryption Standard)**:
    - **Key Generator**: Visualizes generation of 16 round keys from 64-bit hex key.
    - **Full DES Logs**: Complete step-by-step execution logs for every round (Encryption & Decryption).
    - **Smart Workflow**: "To Input" button to instantly transfer ciphertext for verification.
    - Attributes with **visual icons** (📂 , 🔑) for better UX.
- **S-DES (Simplified DES)**:
    - Educational 8-bit block cipher.
    - **Optimized Layout**: Logical flow with Key Generation side-by-side with Subkeys.
    - Full encryption/decryption pipeline.

🛡️ Hashing Algorithms

- **MD5 & Full MD5**: Step-by-step round visualization and full 128-bit digest calculation.
- **SHA-1 & SHA Family**: Generate standard 160-bit hash digests.

🌐 Web Standards

- **JWT (JSON Web Token)**:
    - **Generator**: Create signed tokens with custom payloads.
    - **Verifier**: Verify token integrity and decode claims.
    - **Visualizer**: See the structure of Header, Payload, and Signature.

⚙️ Utility Features

- **File Operations**: Load `.txt` files directly into inputs and save results to disk.
- **Clipboard**: One-click copy buttons (📋).

- **Compact UI**: "Minimized" padding for a sleek, information-dense display.

🚀 Getting Started

Prerequisites

- **Python 3.6+**: Download Python
- **Pillow (PIL)**: Required for handling images and diagrams.
- **PyJWT**: Required for JSON Web Token operations.

```
pip install pillow
pip install PyJWT
```

Installation

1. **Clone the repository** (or download source):

```
git clone https://github.com/Mahmoud-keno/security-toolkit.git
cd security-toolkit
```

2. **Install Dependencies**:

```
pip install pillow
pip install PyJWT
```

🏃 How to Run

1. Navigate to the project directory.
2. Run the application:

```
python security_toolkit.py
```

3. Choose **Codes** to run algorithms or **Explanation** to learn about them.

🖥️ Usage Guide

1. **Landing Page**: Select your mode.
2. **Codes Mode**:
    - **Tabs**: Switch between algorithms (RSA, DES, etc.).

- **Input**: Type text or use 📂 **Load File**.
- **Action**: Click Green/Blue action buttons (e.g., **Encrypt**, **Generate Keys**).
- **Output**: Copy (📋) or Save (💾) your results.

3. **Explanation Mode**:
   - Read detailed breakdowns of each algorithm.
   - View accompanying flowcharts and diagrams on the right panel.

🤝 Contributing

Contributions are welcome!

1. Fork the project.
2. Create your feature branch (`git checkout -b feature/AmazingFeature`).
3. Commit your changes.
4. Push to the branch.
5. Open a Pull Request.

👥 Team Members

- ضحي محمد محمد حنفي
- عمر محمود سعد السيد
- محمود قناوي محمود عبد النبي

📄 License

This project is open source and available under the MIT License.